Florentina Nicolau, Hugues Mounier,
Silviu-Iulian Niculescu

Discretizers and Controllability of Delay Systems

An algebraic-based approach – Appendix B

November 6, 2026

Springer Nature

Contents

B OreModules codes used in Chapter 5 1
B.1 System over R_E 1
B.2 Distributed extension 5
B.3 Discretization 8

References 13

Appendix B
OreModules codes used in Chapter 5

This appendix gathers the complete Maple scripts used, with the package OreModules [1], for the computations of Section 5.2 (see Subsection 5.2.4, where shorter extracts of these scripts are commented). The three scripts below have been run with Maple 2026. They follow the conventions of Subsection 5.2.4: `Dt` stands for $\frac{d}{dt}$, `delta` for $\delta_L = e^{-L\frac{d}{dt}}$, `E` for e^{L} (so that $e^{-L(\frac{d}{dt}-1)}$ = `E*delta`), `dd` for the distributed delay operator $\mathfrak{d}_L$ of Subsection 5.2.1.2, `dh` for δ_h, and $L = Mh$. The computations are performed over the field of rational functions in the parameters (`E`, resp. `h` and `E`). Before running a script, the variable `libname` has to be set so that it contains the path to the OreModules library (see the commented line at the beginning of each script). Each script can then be run in batch mode, e.g., with `maple -q 01_system_Rex.mpl`, or, after deleting the final `quit;`, in an interactive session with `read "01_system_Rex.mpl";`.

The extension modules returned by the command `Exti` are interpreted by means of Propositions 5.1 and 5.2: $\mathrm{Ext}^1_R(N,R) = 0$ if, and only if, the system is torsion-free, and the system is projective if, and only if, $\mathrm{Ext}^i_R(N,R) = 0$ for $1 \leq i \leq \mathrm{gl.dim}(R)$ (OreModules returns the matrix (1), or `undefined`, for the zero module).

B.1 System over $\mathbb{R}\left[\frac{d}{dt}, e^{-L\frac{d}{dt}}\right]$

This section corresponds to Subsections 5.2.1 and 5.2.1.1. The script below computes the minors and the Fitting ideal of Λ_e^{ε}, its torsion submodule, the decoupling change of variables $\xi_1 = x_1 - x_2$, $\xi_2 = x_1 + x_2$, and checks that $\Lambda_e^{\varepsilon}/\mathrm{Tor}(\Lambda_e^{\varepsilon})$ is torsion-free but not projective over $\mathbb{R}\left[\frac{d}{dt}, e^{-L\frac{d}{dt}}\right]$, for $\varepsilon = 1$ and $\varepsilon = -1$.

```
## ======================================================================
## Section 5.2 -- "A more elaborated example"
## Part 1: model, Fitting ideal, torsion / torsion-freeness /
```

```
##         projectivity and decomposition over Rex = R[d/dt, e^{-L d/dt}]
##         (Subsections 5.2.1 and 5.2.1.1 of the book)
##
## Maple conventions:
##   Dt     = d/dt
##   delta  = e^{-L d/dt}   (pure delay, independent of Dt in Rex)
##   E      = e^{L}         (real, strictly positive parameter)
## so that e^{-L(d/dt - 1)} = E*delta.
##
## The coefficients being constant, the Ore algebra defined below is
## the commutative ring Q(E)[Dt, delta], isomorphic to Rex.
## ======================================================================

## Path to the OreModules library (to be adapted to the local installation)
    :
## libname := "/path/to/OreModules", libname:
with(OreModules):
with(LinearAlgebra):
interface(screenwidth = 110):

A := DefineOreAlgebra(diff = [Dt, t], dual_shift = [delta, s],
                      polynom = [t, s], comm = [E]):

## Application of a presentation matrix to a vector of generators
## (Dt and delta commute, so that the formal product is licit).
apply_to := (M, v) -> map(expand, convert(Matrix(M) . Vector(v), list)):

for eps in [1, -1] do

  printf("\n\n#####################################################\n");
  printf("#####   epsilon = %2d                         #####\n", eps);
  printf("#####################################################\n");

  ## --- Presentation matrix of Lambda_e^epsilon (variables x1, x2, u)
  b := 1 + eps*E*delta:            # 1 + epsilon e^{-L(d/dt-1)}
  R := evalm([[Dt, -1, -b],
              [-1, Dt, -b]]):
  printf("\nPresentation matrix P (columns x1, x2, u):\n");
  print(R);

  ## --- 2x2 minors (m_i = minor obtained by deleting column i)
  RM := Matrix(R):
  m := [seq(factor(Determinant(DeleteColumn(RM, i))), i = 1 .. 3)]:
  printf("\nMinors of order 2 [m1, m2, m3]:\n");
  print(m);
```

```
## --- Fitting ideal: gcd of the minors and Groebner basis of the
  quotient
g := factor(gcd(gcd(m[1], m[2]), m[3])):
printf("\nGcd␣of␣the␣minors␣(common␣factor):␣%a\n", g);
q := map(normal, map(z -> z/g, m)):
printf("Fitt(Lambda_e)␣=␣(%a)␣*␣<␣%a␣>\n", g,
       Groebner:-Basis(q, plex(Dt, delta)));

## --- Common zeros of the minors in C^2 (s = Dt, z = delta)
printf("\nCommon␣zeros␣of␣the␣minors␣in␣C^2:\n");
print(solve({op(m)}, {Dt, delta}));

## --- Extension modules (Ext^i(N, D), N = transposed module of Lambda)
##     Ext^1 = 0  <=> torsion-free
##     Ext^1 = Ext^2 = 0 (and Ext^3 = 0) <=> projective
##     (OreModules returns the matrix [1] or "undefined" for the zero
  module)
e1 := Exti(Involution(R, A), A, 1):
printf("\nExt^1:␣presentation␣matrix␣of␣the␣torsion␣submodule\n");
print(e1[1]);
printf("Generators␣of␣the␣torsion␣(rows␣of␣e1[2]␣applied␣to␣[x1,x2,u]):\n
  ");
print(apply_to(e1[2], [x1, x2, u]));
printf("=>␣Lambda_e␣is␣NOT␣torsion-free:␣torsion␣generated␣by␣xi1␣=␣x1␣-␣
  x2,\n");
printf("␣␣␣with␣the␣relation␣(Dt␣+␣1)␣xi1␣=␣0.\n");

e2 := Exti(Involution(R, A), A, 2):
printf("\nExt^2:\n");
print(e2[1]);
e3 := Exti(Involution(R, A), A, 3):
printf("Ext^3:␣%a␣␣␣(undefined/ZERO␣=␣zero␣module)\n", e3[1]);

## ---------------------------------------------------------------
## Decomposition (Subsection 5.2.1.1):
##   xi1 = x1 - x2, xi2 = x1 + x2.
## W: change of variables (xi1, xi2, u) = W (x1, x2, u)
## U: combination of the equations. U and W are invertible over R.
## ---------------------------------------------------------------
W := Matrix([[1, -1, 0], [1, 1, 0], [0, 0, 1]]):
U := Matrix([[1, -1], [1, 1]]):
Rdec := map(expand, U . RM . MatrixInverse(W)):
printf("\nAfter␣the␣change␣of␣variables␣(xi1,␣xi2,␣u):␣U␣P␣W^(-1)␣=\n");
print(Rdec);
printf("=>␣decoupled␣system:␣(Dt+1)␣xi1␣=␣0␣and␣(Dt-1)␣xi2␣-␣2(1␣+␣eps␣E␣
  delta)␣u␣=␣0.\n");
```

```
  ## --- Torsion part: T = [xi1] / ((Dt+1) xi1)
  RT := evalm([[Dt + 1]]):
  printf("\nTor(Lambda_e): presentation %a, Fitt = (%a)\n",
         convert(RT, listlist), Dt + 1);
  printf("Exti(...,1) for T (T is a torsion module, Ext^1 = T):\n");
  print(Exti(Involution(RT, A), A, 1)[1]);

  ## --- Torsion-free part: F = Lambda_e / Tor(Lambda_e) = [xi2, u]
  RF := evalm([[Dt - 1, -2*b]]):
  printf("\nLambda_e / Tor(Lambda_e): presentation (columns xi2, u)\n");
  print(RF);
  printf("Fitt = <%a>\n", Groebner:-Basis([Dt - 1, 2*b], plex(Dt, delta))
    );
  printf("Ext^1 (the matrix [1] presents the zero module => torsion-free):\
    n");
  print(Exti(Involution(RF, A), A, 1)[1]);
  eF2 := Exti(Involution(RF, A), A, 2):
  printf("Ext^2 (0 <=> projective):\n");
  print(eF2[1]);
  printf("Common zero in C^2 of the generators of Fitt: %a\n",
         solve({Dt - 1, b}, {Dt, delta}));
  printf("=> Lambda_e/Tor is torsion-free but NOT projective (hence not
    free)\n");
  printf("   over Rex = R[d/dt, e^{-L d/dt}], for epsilon = %d.\n", eps);

  ## --- Parametrization of the torsion-free part (image of an operator)
  printf("\nParametrization of Lambda_e/Tor: (xi2, u)^T = Q xi, Q =\n");
  print(Exti(Involution(RF, A), A, 1)[3]);

  ## --- Link with the zeros in s (Laplace): z = e^{-L s}, E = e^{L}
  printf("\nRestriction to the curve z = e^{-L s}:\n");
  printf("  1 + eps*E*delta at s = 1: %a\n",
         simplify(subs(E = exp(L), delta = exp(-L*1), b)));
  printf("  (common zero at s = 1 only if epsilon = -1)\n");

end do:

quit;
```

B.2 Distributed extension

This section corresponds to Subsection 5.2.1.2. The script below works in the subring $\mathscr{R}_{\mathfrak{d}} = \mathbb{R}[\frac{d}{dt}, \mathfrak{d}_L]$ of $\mathscr{R}_d$ (Maple name S). For $\varepsilon = -1$, it exhibits the additional torsion element $\xi_2 - 2\mathfrak{d}_L u$; for $\varepsilon = 1$, it checks that the torsion-free part is free and recovers the Bézout coefficients $r_1 = -1/4$, $r_2 = \mathfrak{d}_L/2$ and the basis ω.

```
## ======================================================================
## Section 5.2 -- "A more elaborated example"
## Part 2: distributed extension Lambda_d^epsilon
##         (Subsection 5.2.1.2 of the book, ring R_d)
##
## R_d = R(d/dt)[e^{-L d/dt}, e^{L d/dt}] cap O is not a polynomial ring:
## OreModules cannot compute in it directly. We use its subring
##
##        S = R[d/dt, dd]  contained in R_d,
##        dd = (1 - e^{-L(d/dt - 1)}) / (d/dt - 1)   (distributed delay),
##
## which is a polynomial ring in two indeterminates (d/dt and dd are
## algebraically independent) and which contains Rex, since
##
##        e^{-L(d/dt - 1)} = E*delta = 1 - (Dt - 1)*dd.
##
## Any torsion element found over S remains a torsion element over R_d,
## and a free module over S yields a free module over R_d by extension of
## scalars: the conclusions obtained over S are transported to R_d.
##
## Maple conventions: Dt = d/dt, dd = distributed delay, E = e^{L}.
## ======================================================================

## Path to the OreModules library (to be adapted to the local installation)
    :
## libname := "/path/to/OreModules", libname:
with(OreModules):
with(LinearAlgebra):
interface(screenwidth = 110):

S := DefineOreAlgebra(diff = [Dt, t], dual_shift = [dd, s],
                      polynom = [t, s], comm = [E]):

apply_to := (M, v) -> map(expand, convert(Matrix(M) . Vector(v), list)):

## Check: dd is an entire function of s (no pole at s = 1)
printf("dd(s) = (1 - exp(-L(s-1)))/(s-1), limit at s = 1: %a\n",
       limit((1 - exp(-L*(s - 1)))/(s - 1), s = 1));
```

```
for eps in [-1, 1] do

  printf("\n\n#####################################################\n");
  printf("#####␣␣␣epsilon␣=␣%2d␣␣␣(ring␣S␣=␣R[Dt,␣dd])␣␣␣␣␣␣␣␣#####\n", eps
     );
  printf("#####################################################\n");

  ## 1 + epsilon e^{-L(d/dt-1)} expressed in S
  b := expand(1 + eps*(1 - (Dt - 1)*dd)):
  printf("\n1␣+␣epsilon␣e^{-L(d/dt-1)}␣=␣%a\n", factor(b));

  ## --- Complete system (x1, x2, u)
  R := evalm([[Dt, -1, -b],
              [-1, Dt, -b]]):
  printf("\nPresentation␣matrix␣of␣Lambda_d␣(columns␣x1,␣x2,␣u):\n");
  print(R);
  m := [seq(factor(Determinant(DeleteColumn(Matrix(R), i))), i = 1 .. 3)]:
  printf("Minors␣of␣order␣2:␣%a\n", m);
  printf("Gcd␣of␣the␣minors:␣%a\n", factor(gcd(gcd(m[1], m[2]), m[3])));
  printf("Rank␣of␣Lambda_d␣(=␣3␣-␣rank␣of␣P):␣%a\n", 3 - OreRank(R, S));

  ## --- Torsion
  e1 := Exti(Involution(R, S), S, 1):
  printf("\nExt^1:␣presentation␣of␣the␣torsion␣submodule␣Tor(Lambda_d)\n");
  print(e1[1]);
  printf("Generators␣of␣Tor(Lambda_d):\n");
  print(apply_to(e1[2], [x1, x2, u]));

  ## --- Decomposition (xi1, xi2, u)
  W := Matrix([[1, -1, 0], [1, 1, 0], [0, 0, 1]]):
  U := Matrix([[1, -1], [1, 1]]):
  Rdec := map(factor, U . Matrix(R) . MatrixInverse(W)):
  printf("\nSystem␣in␣the␣variables␣(xi1,␣xi2,␣u):\n");
  print(Rdec);

  ## --- Torsion-free part: parametrization of Lambda_d / Tor(Lambda_d)
  ## e1[3]: matrix Q such that Lambda_d / Tor(Lambda_d) = image of Q
  ## (x = Q xi, with xi arbitrary functions)
  printf("\nParametrization␣of␣Lambda_d␣/␣Tor(Lambda_d):␣(x1,␣x2,␣u)^T␣=␣Q␣
     xi,␣Q␣=\n");
  print(e1[3]);

  if eps = -1 then
    ## ------------------------------------------------------------
    ## epsilon = -1: 1 - e^{-L(d/dt-1)} = (Dt - 1) dd, hence
```

```
  ##   (Dt - 1)(xi2 - 2 dd u) = 0: xi2 - 2 dd u is a torsion element.
  ## Tor(Lambda_d) = [xi1, xi2 - 2 dd u] and Lambda_d / Tor = [u] is
  ## free of rank 1: Lambda_d is not a torsion module.
  ## ------------------------------------------------------------
  tau := expand((x1 + x2) - 2*dd*u):
  printf("\nElement xi2 - 2 dd u = %a\n", tau);
  printf("(Dt - 1)(xi2 - 2 dd u) reduced modulo the equations of the
   system:\n");
  ## Sum of the two equations: (Dt-1)(x1+x2) - 2 b u = 0
  printf("  %a\n", expand((Dt - 1)*tau - (Dt*x1 - x2 - b*u) - (-x1 + Dt*
   x2 - b*u)));
  printf("=> xi2 - 2 dd u is a torsion element, as xi1 = x1 - x2.\n");

  RF := evalm([[Dt - 1, -2*b]]):    # columns xi2, u
  printf("\nPart (xi2, u): %a\n", convert(RF, listlist));
  eF := Exti(Involution(RF, S), S, 1):
  printf("Ext^1 of the part (xi2, u) (nonzero torsion):\n");
  print(eF[1]);
  printf("Corresponding torsion element: %a\n",
         apply_to(eF[2], [xi2, u]));
  printf("Quotient by the torsion: generated by u alone, without relation
   \n");
  printf("=> free of rank 1, with basis u.\n");

else
  ## ------------------------------------------------------------
  ## epsilon = 1: the part (xi2, u) is free over S, hence over R_d.
  ## ------------------------------------------------------------
  RF := evalm([[Dt - 1, -2*b]]):    # columns xi2, u
  printf("\nTorsion-free part (columns xi2, u):\n");
  print(RF);
  for i from 1 to 3 do
    printf("Ext^%d: %a\n", i, Exti(Involution(RF, S), S, i)[1]);
  end do:
  printf("=> Ext^1 = Ext^2 = Ext^3 = 0: projective, hence free (Quillen-
   Suslin)\n");

  ## Bezout equation of the book: r1 m1 + r2 m2 = 1 with
  ##   m1 = -2(1 + e^{-L(d/dt-1)}),  m2 = Dt - 1  (minors of RF)
  m1 := -2*b:  m2 := Dt - 1:
  Rinv := RightInverse(RF, S):
  printf("\nRight inverse of RF (OreModules):\n");
  print(Rinv);
  r20 := Rinv[1, 1]:  r10 := Rinv[2, 1]:
  printf("Bezout coefficients found: r1 = %a, r2 = %a\n", r10, r20);
  printf("Check r1 m1 + r2 m2 = %a\n", expand(r10*m1 + r20*m2));
```

```
    r1 := -1/4:  r2 := dd/2:
    printf("\nCoefficients␣of␣the␣book:␣r1␣=␣-1/4,␣r2␣=␣dd/2\n");
    printf("Check␣r1␣m1␣+␣r2␣m2␣=␣%a\n", expand(r1*m1 + r2*m2));

    ## Basis omega = r1 xi2 - r2 u and expression of xi2, u in terms of
     omega
    printf("\nBasis␣omega␣=␣r1␣xi2␣-␣r2␣u␣=␣%a\n", r1*xi2 - r2*u);
    printf("xi2␣=␣m1␣omega␣=␣(%a)␣omega,␣␣␣u␣=␣-m2␣omega␣=␣(%a)␣omega\n",
           factor(m1), -m2);
    printf("Check:␣(Dt-1)(m1␣omega)␣-␣2␣b␣(-m2␣omega)␣=␣%a\n",
           expand((Dt - 1)*m1 - 2*b*(-m2)));
    printf("Check:␣r1␣(m1␣omega)␣-␣r2␣(-m2␣omega)␣=␣%a␣omega\n",
           expand(r1*m1 + r2*m2));

    ## With OreModules: injective parametrization and left inverse
    PF := Exti(Involution(RF, S), S, 1)[3]:
    printf("\nParametrization␣of␣the␣free␣part␣computed␣by␣OreModules:␣(xi2
     ,␣u)^T␣=␣Q␣xi,␣Q␣=\n");
    print(PF);
    Q := evalm([[m1], [-m2]]):
    printf("Parametrization␣of␣the␣book␣Q␣=␣[m1,␣-m2]^T:␣RF␣Q␣=␣%a\n",
           map(expand, convert(evalm(RF &* Q), listlist)));
    Linv := LeftInverse(Q, S):
    printf("Left␣inverse␣of␣Q␣(=>␣omega␣as␣a␣combination␣of␣xi2,␣u):␣%a\n",
           convert(Linv, listlist));
    printf("Check␣Linv␣Q␣=␣%a\n",
           map(expand, convert(evalm(Linv &* Q), listlist)));
  end if:

end do:

quit;
```

B.3 Discretization

This section corresponds to Subsection 5.2.2. The script below treats the discretized system $\mathbb{D}_{B,h}(\Lambda^{\varepsilon})$ over $\mathbb{R}[\delta_h] = \mathbb{R}[\delta_h]$, for $\varepsilon = \pm 1$ and $M = 2, 3$: minors, Fitting ideal, torsion, Bézout coefficients (computed both with `RightInverse` and with the extended Euclidean algorithm `gcdex`), and exceptional values of h. The coefficients

`r1book`, `r2book` are those of an earlier version of the book, whose coefficient $r_{1\mathbb{D}}$ had the wrong sign; the corrected values are those given in Subsections 5.2.2 and 5.2.4.

```
## ======================================================================
## Section 5.2 -- "A more elaborated example"
## Part 3: discretization by the backward difference scheme
##         (Subsection 5.2.2 of the book, ring Rs = R[delta_h])
##
## Maple conventions:
##   dh = delta_h (delay of one period h),  L = M h,
##   E  = e^{L} (treated as a parameter; E = exp(M h) in practice).
## The OreModules computations are performed over Q(h, E)[dh], i.e., for
## generic values of h and E; the exceptional values are examined at
## the end.
## ======================================================================

## Path to the OreModules library (to be adapted to the local installation)
    :
## libname := "/path/to/OreModules", libname:
with(OreModules):
with(LinearAlgebra):
interface(screenwidth = 110):

Rs := DefineOreAlgebra(dual_shift = [dh, k], polynom = [k], comm = [h, E]):

apply_to := (M, v) -> map(expand, convert(Matrix(M) . Vector(v), list)):

## Bezout coefficients given in an earlier version of the book (M = 2 and
## M = 3). The script shows that r1 must be replaced by -r1 for the Bezout
## equation  -2h(1 + eps E dh^M) r1 + (1 - h - dh) r2 = 1  to hold; the
## corrected values (those returned by RightInverse and gcdex) are the
## ones given in Subsections 5.2.2 and 5.2.4 of the book.
r1book := table():  r2book := table():
r1book[2] := 1/(2*h*(eps*E*(h - 1)^2 + 1)):
r2book[2] := eps*E*(dh - h + 1)/(eps*E*(h - 1)^2 + 1):
r1book[3] := -1/(2*h*(eps*E*(h - 1)^3 - 1)):
r2book[3] := -E*eps*(dh^2 + (1 - h)*dh + (1 - h)^2)/(eps*E*(h - 1)^3 - 1):

for eps in [1, -1] do
for M in [2, 3] do

  printf("\n\n#######################################################\n");
  printf("#####   epsilon = %2d,   M = %d                    #####\n", eps,
      M);
  printf("#######################################################\n");
```

```
b := 1 + eps*E*dh^M:            # discretization of 1 + epsilon e^L
   delta_L

## --- Discretization of the original model (x1, x2, u):
##     (1 - dh) x1 = h (x2 + b u),  (1 - dh) x2 = h (x1 + b u)
Rx := evalm([[1 - dh, -h, -h*b],
             [-h, 1 - dh, -h*b]]):
printf("\nDiscretized␣original␣model␣(columns␣x1,␣x2,␣u):\n");
print(Rx);

## Same change of variables as in continuous time
W := Matrix([[1, -1, 0], [1, 1, 0], [0, 0, 1]]):
U := Matrix([[1, -1], [1, 1]]):
R := map(expand, U . Matrix(Rx) . MatrixInverse(W)):
printf("In␣the␣variables␣(xi1,␣xi2,␣u):␣P_D␣=\n");
print(R);

## --- Minors and Fitting ideal
m := [seq(factor(Determinant(DeleteColumn(R, i))), i = 1 .. 3)]:
printf("\nMinors␣of␣order␣2␣[m1D,␣m2D,␣m3D]:␣%a\n", m);
g := factor(gcd(gcd(m[1], m[2]), m[3])):
printf("Fitt␣=␣(%a)␣*␣(%a,␣%a)\n", g, factor(m[2]/g), factor(m[3]/g));

## --- Torsion
e1 := Exti(Involution(convert(R, matrix), Rs), Rs, 1):
printf("\nExt^1:␣presentation␣of␣Tor(D(Lambda))\n");
print(e1[1]);
printf("Generators:␣%a\n", apply_to(e1[2], [xi1, xi2, u]));
printf("(the␣second␣generator␣is␣zero:␣its␣relation␣is␣1␣=␣0,␣Tor␣=␣[xi1
   ])\n");

## --- Torsion-free part (xi2, u): free (Rs is a principal ideal domain)
m1 := -2*h*b:  m2 := 1 - h - dh:
RF := evalm([[m2, m1]]):         # columns xi2, u
eF := Exti(Involution(RF, Rs), Rs, 1):
printf("\nPart␣(xi2,␣u):␣%a\n", convert(RF, listlist));
printf("Ext^1␣=␣%a␣␣(zero␣module␣=>␣torsion-free,␣hence␣free␣over␣Rs)\n",
       convert(eF[1], listlist));

## Bezout coefficients with OreModules
Rinv := RightInverse(RF, Rs):
r20 := normal(Rinv[1, 1]):  r10 := normal(Rinv[2, 1]):
printf("\nBezout␣(OreModules,␣right␣inverse):\n");
printf("␣␣r1D␣=␣%a\n␣␣r2D␣=␣%a\n", factor(r10), factor(r20));
printf("␣␣r1D␣m1D␣+␣r2D␣m2D␣=␣%a\n", normal(r10*m1 + r20*m2));
```

```
  ## Bezout coefficients with the extended Euclidean algorithm (in dh)
  pg := gcdex(m1, m2, dh, 'c1', 'c2'):
  printf("Bezout (gcdex): gcd = %a\n", pg);
  printf("  r1D = %a\n  r2D = %a\n", factor(c1/pg), factor(c2/pg));

  ## Check of the formulas of the book
  printf("\nFormulas of the earlier version of the book (M = %d):\n", M);
  printf("  r1D = %a\n  r2D = %a\n", r1book[M], r2book[M]);
  printf("  r1D m1D + r2D m2D              = %a\n",
         factor(normal(r1book[M]*m1 + r2book[M]*m2)));
  printf("  (-r1D) m1D + r2D m2D           = %a\n",
         normal(-r1book[M]*m1 + r2book[M]*m2));
  printf("=> the formulas above satisfy the Bezout equation with\n");
  printf("   r1D replaced by -r1D (or with m1D = +2h(1 + eps E dh^M)).\n");

  ## Basis omega_D = r1D xi2 - r2D u (with the OreModules coefficients)
  printf("\nBasis omega_D = r1D xi2 - r2D u, with xi2 = m1D omega_D, u = -
     m2D omega_D\n");
  printf("  check of the relation: %a\n",
         expand(m2*m1 + m1*(-m2)));
  printf("  check r1D m1D + r2D m2D: %a\n", normal(r10*m1 + r20*m2));

  ## --- Exceptional values of h: common zero dh = 1 - h
  cond := subs(dh = 1 - h, b):
  printf("\nFreeness condition (no common zero): %a <> 0\n", cond);
  sol := fsolve(subs(E = exp(M*h), cond), h = 0.001 .. 10):
  if type(sol, numeric) then
    printf("With E = exp(M h): %a = 0 for h = %a (h > 1, outside the usual\
     n", cond, sol);
    printf("  sampling range); for 0 < h < 1 the module is free.\n");
  else
    printf("With E = exp(M h): no solution h in ]0, 10] => free for these h
     .\n");
  end if:

end do:
end do:

quit;
```

References

1. F. Chyzak, A. Quadrat, and D. Robertz. OreModules: a symbolic package for the study of multidimensional linear systems. MAPLE package, `https://who.rocq.inria.fr/Alban.Quadrat/OreModules/index.html`.